

Detecting and Mapping Traffic Signs from Road Photographs: A Held-Out Evaluation on ARTSv2

Muhammad Jawad
jawadkhangigyani@gmail.com

Abstract

I built a pipeline that finds traffic signs in road survey photographs, places each sign on a map, and merges repeated sightings of one signpost into one database row. I used the ARTSv2 dataset from Vermont [1]. I did not propose a new method: I combined a YOLO11n detector, a small classifier that re-reads look-alike signs, a single-view projection from the camera heading, and clustering by position. I split the data by 500 m map tiles and removed every training photograph that shows a validation or test sign, because in the split that ships with the dataset 86% of the test sign ids also appear in training. On 2,502 held-out test photographs the detector reaches **0.723 mAP50**. On 18 held-out road sequences with 366 signposts, the pipeline finds **87.7%** of them and places them with a median error of **4.7 m**, 56% lower than the camera GPS (10.7 m). The classifier raises accuracy on look-alike sign families from 78.7% to 94.3%, and because rows are formed per class, it also raises the deduplication score from 49.5% to 67.9%. Deduplication is still the main weakness: 49 signposts are split across rows and 54 rows merge two or more signposts. Code, weights and results are at <https://github.com/JawadGigyani/Geo-Localization-of-Traffic-Signs>.

1 Introduction

Road agencies keep inventories of their traffic signs: what each sign is and where it stands. Transportation departments collect millions of street images a year to manage these assets [1]. Turning the images into an inventory takes three steps: find each sign, place it on the map, and count each physical sign once, even when it appears in many photographs.

The last step is harder than it sounds. In ARTSv2, frames were taken from the survey video at about one-second intervals [1], so a sign moves a long way between frames, while standard multi-object trackers assume that objects move only a little between frames [1]. Earlier work solved the mapping step in different ways. Campbell et al. detected signs in Google Street View images and mapped them with GIS tools [2]. Krylov et al. combined monocular depth estimates with triangulation [3]. Wilson et al. trained a detector that predicts each sign’s offset from the camera, together with a learned tracker [1].

I built a simple pipeline from standard parts and evaluated it on roads it never saw. I did not propose a new method. My goal was to measure how well a plain approach works and where it fails:

- I built and evaluated the full pipeline, from photographs to one row per signpost, and regenerated every model and pipeline number from the released weights; the dataset statistics come from the training notebook (Section 5).
- I built a split in which no sign id is shared between training and test, and measured that the dataset’s own split shares 4,424 of its 5,162 test sign ids with training (Section 2).
- I measured where the pipeline fails: look-alike sign families, deduplication in dense sequences, range estimation, and video trackers on photographs that are metres apart (Section 6).

2 Dataset

ARTSv2 [1] was recorded by a vehicle with a roof camera driving across Vermont. Every annotation has a class, a bounding box, the sign’s own GPS position, its side of the road, an assembly flag and a sign id that

Table 1. The geographic split. A signpost groups ids of the same class within 2 m (Section 4). Photographs and instances: `data/dataset_summary.json`; ids and signposts: `results/split_check.json`.

Split	Photographs	Sign instances	Sign ids	Signposts
Training	10,332	17,594	4,051	3,554
Validation	2,301	3,867	819	758
Test	2,502	4,518	1,136	986

stays the same in every photograph of that sign. I used the `challenging_dom` folder of the release: 19,914 photographs at 1920×1080 , 35,979 annotations in 171 classes and 7,773 distinct sign ids. Every photograph has the camera’s latitude, longitude and heading. The signs are small: the median annotation is 26 pixels on its shorter side, 60.5% are under 32 pixels and 20.4% are under 16 pixels. The classes follow a long tail. The 50 most common classes hold 75.1% of the annotations, and I kept only those, because the classes below them have too few examples; the rarest kept class has 204 annotations.

Split. Neighbouring photographs show the same signs, so a random split puts the same signpost in training and test. This is a known problem for data with spatial structure [4] and a common form of leakage [5]. The split that ships with ARTSv2 has it: 4,424 of the 5,162 sign ids in its test split (86%) also appear in training, and 92.5% of its test photographs show at least one such sign. So I grouped photographs into tiles of 0.005 degrees (about 500 m) by camera position, 1,190 tiles in total, and assigned whole tiles to training, validation and test (15% of tiles each for validation and test, fixed random seed). A sign near a tile border can still be seen from two tiles: 302 sign ids appeared in training and in validation or test. I removed every training photograph that shows a validation or test sign id (704 photographs) and every validation photograph that shows a test sign id (47). Photographs without any of the 50 kept classes were dropped. Table 1 shows the result. No test sign id appears in training; counted by signpost instead, 1 of the 986 test signposts also has an id in training.

Road sequences. To evaluate the map I need whole stretches of road. I exported the 30 largest test tiles with at least 20 photographs (out of 42) as road sequences, 1,108 photographs in total. The first 12 by name (40%, 462 photographs) are used only to fit the camera constants. The other 18 (646 photographs, 451 sign ids, 366 signposts) are used for every pipeline number in this report. Tile names start with a latitude index, so all 12 calibration tiles lie south of the 18 evaluation tiles.

3 Method

3.1 Detector

I used YOLO11n [6], the smallest model of the YOLO family [7], with 2.6 million parameters, starting from the weights Ultralytics trained on COCO [8]. I trained it at an input size of 1280 pixels instead of the default 640. Networks built for large objects do not work well for signs that fill a small part of the image [9], and at 640 pixels the median sign would shrink from 26 to about 9 pixels, just above the network’s smallest feature stride of 8 pixels. I trained for 40 epochs with batch size 8 on a Google Colab T4 GPU, with the default Ultralytics augmentation (mosaic, scaling, colour changes and horizontal flips). Ultralytics chose AdamW [10] with a learning rate of 0.000185, and kept the checkpoint with the best validation score. Early stopping with a patience of 12 epochs never triggered. Colab sessions ended five times during training, and each time I resumed from the last checkpoint; training started on Ultralytics 8.4.118 and finished on 8.4.120.

3.2 Crop classifier for look-alike signs

Of the 50 classes, 34 belong to 11 families with more than one member, such as M3 and R2. Signs in one family can differ only in small details. In the MUTCD, M3-1 to M3-4 are cardinal-direction plaques that carry the word NORTH, EAST, SOUTH or WEST [11]; I checked crops from the dataset, and its M3-2 and M3-4

plaques do read EAST and WEST (for example ARTSv2 frames 13000000066065 and 13000000101530). In the demo sequence, the first view of one EAST plaque is a box of 18 by 8 pixels (frame 13000000041064). So the detector picks the family and a second model picks the variant. The second model is a YOLO11n-cls classifier with 1.6 million parameters, pretrained on ImageNet [12, 6]. It reads the detected box, padded by 15% and resized to 96 pixels. Its answer is restricted to the members of the family the detector chose, because it was trained only to tell variants apart, not families. It replaces the detector’s label only if its renormalised confidence is at least 0.8. I trained it on 11,232 crops cut around ground-truth boxes in training photographs, with the same 15% padding so the framing looks like a predicted box, and chose the epoch on 2,173 crops from validation photographs. Early stopping ended training after 36 of 40 epochs and kept epoch 26.

3.3 Placing each sign on the map

Each photograph has the camera’s GPS position and heading. I treat the camera as a pinhole camera [13]. For a frame W pixels wide, a horizontal field of view θ and a box with centre c_x and width w :

$$f = \frac{W/2}{\tan(\theta/2)}, \quad \text{bearing} = \text{heading} + \arctan \frac{c_x - W/2}{f}, \quad \text{distance} = \frac{k f}{w}, \quad (1)$$

with the distance clamped to 2–200 m. The constant k stands for the typical physical width of a sign. I fitted θ and k on the 12 calibration sequences, using 771 detections that match a labelled sign: θ by a grid search from 40 to 140 degrees in 5-degree steps that minimises the squared bearing error, and then k by least squares. This gave $\theta = 80^\circ$ and $k = 0.8807$ m. Both are stored with the weights and used unchanged on the evaluation sequences.

3.4 Merging repeated sightings

Video trackers such as BoT-SORT [14] and ByteTrack [15] link detections between frames mainly by box overlap. Here the camera moves several metres between photographs (Section 5), so two views of a sign rarely overlap. Instead I merge sightings on the map. I sort the detections of a sequence by box width, widest first. Each detection joins the first existing row of the same class within 60 m of that row’s position, or else starts a new row. A row keeps the position projected from its widest box, which is normally the closest view and the most reliable range estimate. I do not triangulate across views. The 327 rows of the evaluation sequences are stored in PostgreSQL with PostGIS and shown on a web map through a FastAPI service.

4 Evaluation Protocol

Models. I report the detector’s mAP50, mAP50-95 [8], precision and recall on the 2,502 test photographs. The validation split chose the checkpoint, so validation numbers are only a check. The classifier’s top-1 accuracy on the 2,173 validation crops is also a validation number, since those crops chose its epoch. In the pipeline, a detection is matched to a labelled box in the same photograph if their IoU is at least 0.3, and its label is checked against the true class.

Signposts. ARTSv2 ids are not a clean key for a physical sign. On the demo sequence, three ids are copies of another id of the same class at exactly the same coordinates, for example 0_D1-1_44.2461... and 1_D1-1_44.2461... So I treat ids of the same class within 2 m as one signpost, which turns the 451 ids of the evaluation sequences into 366 signposts. Only the 50 trained classes are in this ground truth.

Map metrics. A signpost is *found* if at least one detection inside any row matches it, and recall is found divided by all signposts. A found signpost is *fragmented* if it appears in more than one row, and a row is *merged* if it contains more than one signpost. The deduplication score is (found – fragmented – merged)/found. A strict one-to-one match is a signpost with exactly one row that holds nothing else. The position error is the great-circle distance between a row’s position and the true position of the sign its widest box matches; rows whose widest box matches no labelled sign are not scored (249 of 327 rows are). The baseline is the camera’s

Table 2. Results on held-out data. Detector: `results/model_metrics.json`. Pipeline: 18 evaluation sequences, `results/batch_evaluation_stage1.json` (detector labels) and `_stage2.json` (with the classifier). Brackets: 95% bootstrap interval over sequences. Position error is scored on 273 and 249 rows.

<i>Detector, test split (2,502 photographs)</i>		
mAP50 / mAP50-95		0.723 / 0.495
Precision / recall		0.739 / 0.677
<i>Pipeline, 18 evaluation sequences</i>	Detector labels	With classifier
Classification, all matched detections (1,223)	85.5%	94.8%
Classification, look-alike families (723)	78.7%	94.3%
Rows produced	342	327
Signposts found (of 366)	87.7% [82.9, 92.0]	87.7% [82.9, 92.0]
Fragmented signposts / merged rows	92 / 70	49 / 54
Deduplication score	49.5% [37.6, 60.6]	67.9% [62.6, 76.6]
Strict one-to-one signposts	123	160
Position error, median / p90 (m)	4.96 / 18.0	4.73 / 17.2
Camera GPS baseline, median (m)	11.0	10.7

GPS position for the same view. Counts are pooled over the 18 sequences. For recall and deduplication I add 95% intervals from a bootstrap over sequences (10,000 resamples), which I computed for this report from the saved per-sequence counts.

5 Results

Table 2 shows the main results. On the test split the detector reaches 0.723 mAP50 and 0.495 mAP50-95; on validation it reaches 0.745 and 0.534. The classifier reaches 96.6% top-1 on the validation crops. With the classifier, the pipeline finds 321 of the 366 signposts (87.7%) and places the scored rows with a median error of 4.73 m and a 90th percentile of 17.2 m. The camera GPS gives a median of 10.7 m, so projection reduces the median error by 56%. Strict one-to-one matching holds for 160 of the 366 signposts (43.7%).

The classifier helps beyond its own task. It cuts the classification error on look-alike families from 21.3% to 5.7%, a drop of 73%. Rows are formed per class, so wrong labels used to split one signpost into several rows. With the classifier, fragmented signposts drop by 47% (from 92 to 49), merged rows by 23% (from 70 to 54), and the deduplication score rises from 49.5% to 67.9%. Recall does not change, because the classifier does not add or remove detections.

Video trackers. On the demo sequence, one of the 18 evaluation sequences (69 photographs), consecutive photographs are a median 8.08 m apart. The BoT-SORT and ByteTrack implementations in Ultralytics give an identity to only 8 and 7 of its 103 detections. Position clustering gives every detection an identity by construction and forms 22 rows; whether those rows are right is what the deduplication score measures. On this sequence the median position error is 3.0 m, against 7.3 m for the camera GPS.

Published results. Wilson et al. report detection mAP at IoU 0.5 on their test portion of ARTSv2 (their Table 2), and an end-to-end benchmark that counts a predicted sign within 15 m of a real sign as a true positive (their Table 4) [1]. Their split, classes and matching rule differ from mine, so I do not compare numbers.

6 Failure Analysis

6.1 Look-alike signs

Figure 1(a) shows the detector’s test AP50 for each class against its number of training instances. The number of examples does not explain the weak classes. VR-132 and E5-1 have only 113 and 125 training instances

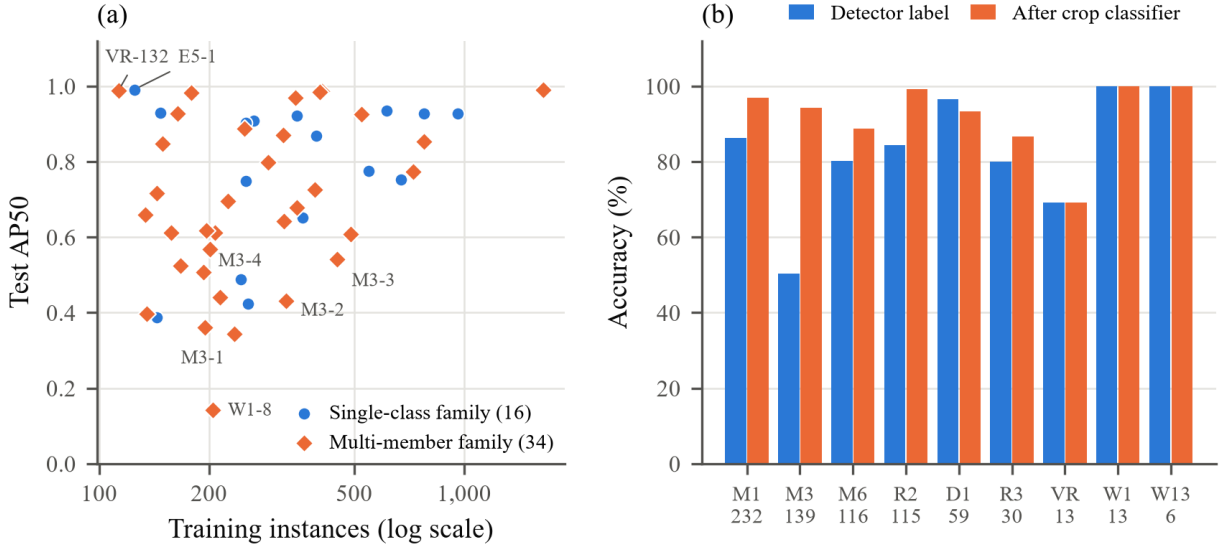


Figure 1. (a) Detector AP50 per class on the test split against training instances (`results/model_metrics.json`, `data/dataset_summary.json`). (b) Classification accuracy of matched detections in the nine look-alike families that occur in the evaluation sequences, before and after the classifier; numbers under the family names are matched detections (`results/batch_evaluation_stage2.json`).

and reach AP50 of 0.988 and 0.991, while the four M3 classes have 193 to 448 and stay between 0.360 and 0.542. Six of the nine classes with AP50 below 0.5 belong to multi-member families, and the median AP50 is 0.687 for classes in such families against 0.885 for the others. Per detection, the detector labels only 50.4% of matched M3 detections correctly, and the classifier raises this to 94.2% (Figure 1b). The classifier is not always right: on the D1 family it lowers accuracy from 96.6% to 93.2%. Its average confidence on its 25 wrong picks is 0.80, the same value as the threshold, against 0.99 on its 679 right picks. I did not measure how many wrong picks the threshold removes.

6.2 Deduplication in dense sequences

After the classifier, 49 signposts are still split across rows and 54 rows merge two or more signposts. A split creates a duplicate in the inventory, and a merge hides a sign. Figure 2 shows that these errors are not spread evenly: the deduplication score of single sequences ranges from 0.25 to 1.0. The largest sequence holds 108 of the 366 signposts (29.5%) but 23 of the 54 merged rows (42.6%), and scores 0.606. My hypothesis is that dense sign assemblies put several signs of the same class within 60 m of each other, so a fixed radius merges them. I have not tested this.

6.3 Range is the weaker half of the position

On the demo sequence I split each row's error into the part along the camera's line of sight and the part across it. The median error is 2.26 m along the line of sight and 1.07 m across it (21 scored rows), so the distance estimate from box width is weaker than the direction. One likely reason is that a single constant k stands for the width of every sign, while real signs differ in size. I have not tested this, and I measured the split on one sequence only.

6.4 Calibration and misses

Fitting the camera constants on the demo sequence alone gives a field of view of 95 degrees (98 matches), against 80 degrees from the 12 calibration sequences. One sequence does not pin down the field of view,

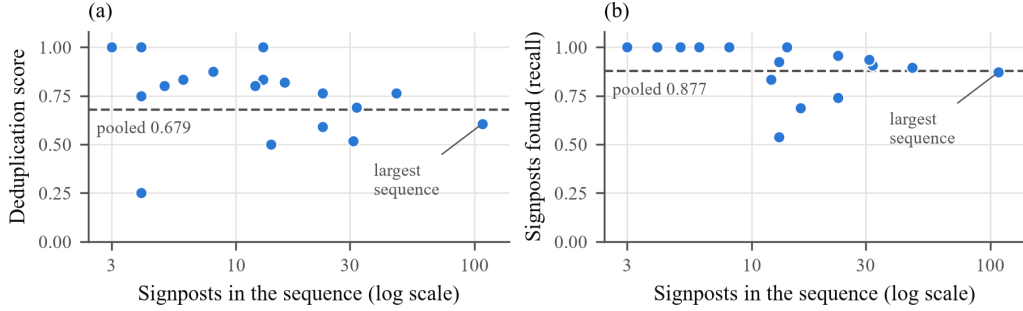


Figure 2. (a) Deduplication score and (b) recall of each evaluation sequence against its number of signposts. Dashed lines: pooled values. Source: `results/batch_evaluation_stage2.json`.

which is why I fit on many sequences and keep them apart from the evaluation. Finally, 45 of the 366 signposts (12.3%) are never found. I have not broken these misses down by distance, size or number of views, so I cannot say why they are missed.

7 Limitations

I trained each model once, so I cannot give error bars for the models; the bootstrap intervals cover only recall and deduplication, because per-row position errors were not saved. Apart from switching the crop classifier off (Table 2) and replacing position clustering with video trackers on one sequence, I did not run ablations. In the reported run I did not tune the 60 m radius or the 0.8 classifier threshold, and I did not test whether the default horizontal flips, which mirror the text on signs, hurt the text-based classes. Only 50 of 171 classes are supported, and the evaluation ground truth contains only those 50, so recall says nothing about other signs. A found signpost means a detection landed on it, not that its class is right. All data comes from one US state and one camera setup. The calibration and evaluation tiles lie in different latitude bands, which is a fair hold-out, but not a test on other cameras. The pipeline reads the camera heading from the files my converter writes, so ordinary geotagged photographs need an adapter. Finally, the evaluation has 366 signposts in 18 sequences, and the largest sequence holds 29.5% of them.

8 Conclusion

A standard detector, a small family-restricted classifier, single-view projection and clustering by position turn ARTSv2 road photographs into a sign map. On held-out roads the pipeline finds 87.7% of signposts with a median position error of 4.7 m, 56% lower than the camera GPS. Look-alike sign families cause most label errors, and fixing them also fixes many deduplication errors, because rows are formed per class. Deduplication remains the weakest part, especially in dense sequences, and range is less accurate than direction.

Reproducibility

Everything is in the repository (<https://github.com/JawadGigyani/Geo-Localization-of-Traffic-Signs>).

- Dataset statistics, the official-split audit, training logs, validation and test mAP: `notebooks/01_train_yolo_colab_G1_G2_RUN.ipynb` (cells B1–B7, C1, D2, D3, E1, G1, G2) and `data/dataset_summary.json`.
- Per-epoch training logs, training sessions, best epochs and early stopping, read from the released weights: `results/detector_training_log.csv`, `results/classifier_training_log.csv` and `results/training_summary.json` (`scripts/12_training_logs.py`).

- Detector and classifier metrics: `results/model_metrics.json` (`scripts/08_evaluate_models.py`).
- Pipeline metrics: `results/batch_evaluation_stage1.json`, `results/batch_evaluation_stage2.json`, `results/sequences_manifest.json` and `weights/calibration.json` (`scripts/07_batch_evaluate.py`).
- Split check: `results/split_check.json` (`scripts/05_export_sequences.py`).
- Tracker baselines, photo spacing, single-sequence fit, line-of-sight errors, duplicate ids, look-alike plaque examples: `results/sequence_checks.json` (`scripts/11_sequence_checks.py`).
- Figures, percent changes and bootstrap intervals: `results/figures/` and `results/derived_numbers.json` (`scripts/13_report_figures.py`).
- `notebooks/02_reproduce_on_colab.ipynb` re-runs scripts 05, 08, 07 (with and without the classifier) and 11 from the released weights; in the committed run every check passes.

References

- [1] D. Wilson, T. Alshaabi, C. Van Oort, X. Zhang, J. Nelson, and S. Wshah. Object tracking and geo-localization from street images. *Remote Sensing*, 14(11):2575, 2022.
- [2] A. Campbell, A. Both, and Q. Sun. Detecting and mapping traffic signs from Google Street View images using deep learning and GIS. *Computers, Environment and Urban Systems*, 77:101350, 2019.
- [3] V. A. Krylov, E. Kenny, and R. Dahyot. Automatic discovery and geotagging of objects from street view imagery. *Remote Sensing*, 10(5):661, 2018.
- [4] D. R. Roberts, V. Bahn, S. Ciuti, et al. Cross-validation strategies for data with temporal, spatial, hierarchical, or phylogenetic structure. *Ecography*, 40(8):913–929, 2017.
- [5] S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.
- [6] G. Jocher and J. Qiu. Ultralytics YOLO11, version 11.0.0. <https://github.com/ultralytics/ultralytics>, 2024.
- [7] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *CVPR*, pages 779–788, 2016.
- [8] T.-Y. Lin, M. Maire, S. Belongie, et al. Microsoft COCO: Common objects in context. In *ECCV*, pages 740–755, 2014.
- [9] Z. Zhu, D. Liang, S. Zhang, X. Huang, B. Li, and S. Hu. Traffic-sign detection and classification in the wild. In *CVPR*, pages 2110–2118, 2016.
- [10] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.
- [11] Federal Highway Administration. *Manual on Uniform Traffic Control Devices*, 2009 edition, Section 2D.15. <https://mutcd.fhwa.dot.gov/hm/2009/part2/part2d.htm>.
- [12] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, pages 248–255, 2009.
- [13] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2nd edition, 2004.
- [14] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. BoT-SORT: Robust associations multi-pedestrian tracking. arXiv:2206.14651, 2022.
- [15] Y. Zhang, P. Sun, Y. Jiang, et al. ByteTrack: Multi-object tracking by associating every detection box. In *ECCV*, LNCS 13682, pages 1–21, 2022.